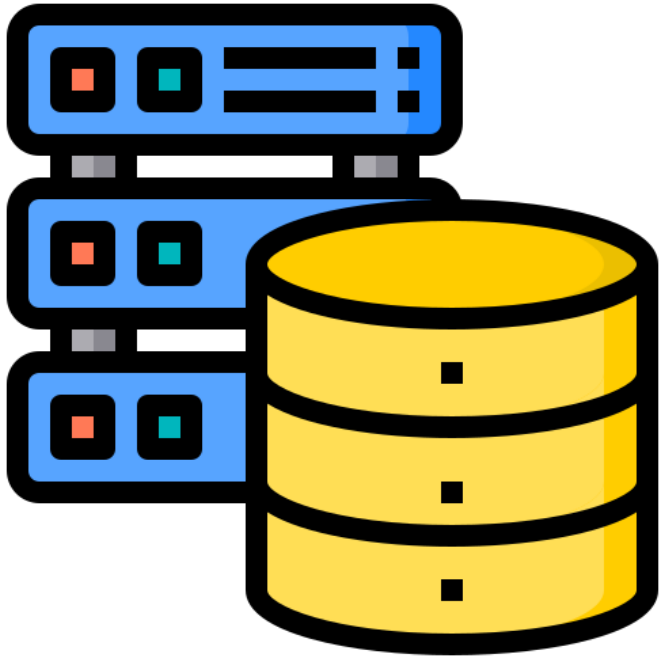
LocalHost ACADEMY

SEMAINE 11 · JOUR 1 — PYTHON & BASES DE DONNÉES

Connecter Python aux bases de données SQL

Ouvrir une connexion SQLite ou PostgreSQL, exécuter des requêtes et afficher les articles du Blog avec leurs auteurs.



Yann Fonkoue

Ingénieur logiciel | Ingénieur IA

Objectifs du jour

1. Expliquer le rôle du pilote, de la connexion et du curseur
2. Ouvrir une base SQLite avec le module intégré sqlite3
3. Connecter Python à PostgreSQL avec psycopg2
4. Exécuter un SELECT et récupérer ses résultats
5. Distinguer fetchone(), fetchall() et le parcours du curseur
6. Afficher les articles du Blog avec le nom de leur auteur

Fil rouge : transformer une requête SQL en un affichage Python lisible.

Pourquoi connecter Python à SQL ?

- La base conserve les données du Blog entre deux exécutions du programme.
- Python demande à la base les informations utiles avec une requête SQL.
- Le pilote traduit les échanges entre le script et le moteur de base de données.
- Le script transforme les résultats en affichage, calcul ou réponse d’API.
- La même démarche servira ensuite dans une application Django ou FastAPI.

Idée clé : SQL sélectionne les données, Python organise leur utilisation.

Requête SQL et traitement Python

Côté base de données

- SELECT choisit les colonnes utiles.
- WHERE filtre les lignes.
- JOIN associe posts et users.
- ORDER BY fixe l’ordre des résultats.

Côté script Python

- connect() ouvre une session.
- execute() transmet la requête.
- fetchone() ou fetchall() lit les lignes.
- Une boucle affiche les valeurs reçues.

Les quatre éléments d’un accès SQL

Pilote	Module qui relie Python au moteur : sqlite3 pour SQLite, psycopg2 pour PostgreSQL.
Connexion	Session ouverte vers un fichier SQLite ou un serveur PostgreSQL.
Curseur	Objet qui exécute SQL et permet de lire les lignes du résultat.
Résultat	Une ligne est souvent un tuple. Plusieurs lignes forment une liste de tuples.

Le cycle d’une requête depuis Python

- Importer le pilote et ouvrir la connexion vers la bonne base.
- Créer un curseur avec conn.cursor().
- Exécuter la requête avec cur.execute(sql, paramètres).
- Lire les lignes avant de réutiliser ce curseur pour une autre requête.
- Fermer le curseur et la connexion, même en cas d’erreur.

Réflexe : utiliser finally pour libérer la connexion à la fin du traitement.

Les méthodes à connaître

- **connect() :** ouvre une connexion, qui doit être fermée après utilisation.
- **cursor() :** crée l’objet utilisé pour exécuter une instruction SQL.
- **execute() :** transmet une instruction et ses éventuels paramètres.
- **fetchone() :** lit la ligne suivante ou renvoie None si le résultat est épuisé.
- **fetchall() :** lit les lignes restantes ou renvoie une liste vide.

À retenir : après fetchone(), fetchall() ne contient plus la ligne déjà lue.

COMPARAISON

SQLite vs PostgreSQL depuis Python

SQLite

- sqlite3 est inclus avec Python.
- La base est un fichier : blog.db.
- Marqueur de valeur : ? avec un tuple.
- connect() crée le fichier s’il manque.

PostgreSQL

- Installer psycopg2-binary pour l’atelier.
- Un serveur et une base blog sont requis.
- Marqueur de valeur : %s avec un tuple.
- Le compte doit avoir les droits nécessaires.

Lire et exploiter les résultats

Une ligne	row = cur.fetchone() retourne un tuple, par exemple (1, "Python et SQL"), ou None.
Plusieurs	rows = cur.fetchall() retourne une liste de lignes, puis [] si aucune ne reste.
Colonnes	for post_id, title in rows décompose chaque tuple dans l'ordre du SELECT.
Cas vide	Tester row is None ou if not rows avant de lire les données ou de les afficher.

Mini-système : lire les articles du Blog

- La table users contient les colonnes id et name.
- La table posts contient id, title, content et user_id.
- Chaque article référence un auteur avec posts.user_id = users.id.
- Le script affiche l’identifiant, le titre et le nom de l’auteur.
- Il doit gérer un résultat vide et fermer correctement la connexion.

Résultat attendu : trois articles ordonnés, associés à Amina ou à Paul.

SQLite : connexion et jointure

Le bloc finally ferme la connexion après la lecture des articles.

```
import sqlite3
conn = sqlite3.connect("blog.db")
try:
    cur = conn.cursor()
    cur.execute("""SELECT p.id, p.title, u.name
        FROM posts p JOIN users u ON u.id = p.user_id
        ORDER BY p.id""")
    for post_id, title, author in cur.fetchall():
        print(f"{post_id}. {title} — {author}")
    cur.close()
finally:
    conn.close()
```

fetchone() : rechercher un article

Le paramètre représente une valeur. La virgule crée un tuple à un élément.

```
import sqlite3
conn = sqlite3.connect("blog.db")
try:
    cur = conn.cursor()
    cur.execute("SELECT title FROM posts WHERE id = ?", (1,))
    row = cur.fetchone()
    if row is None:
        print("Article introuvable")
    else:
        print(row[0])
finally:
    conn.close()
```

PostgreSQL : même logique de lecture

BLOG_DSN contient les paramètres de connexion définis dans l’environnement.

```
import os, psycopg2
conn = psycopg2.connect(os.environ["BLOG_DSN"])
try:
    with conn.cursor() as cur:
        cur.execute("""SELECT p.title, u.name
            FROM posts p JOIN users u ON u.id = p.user_id
            WHERE u.id = %s ORDER BY p.id""", (1,))
        for title, author in cur.fetchall():
            print(title, author)
finally:
    conn.close()
```

EXERCICE

Pratique guidée : le lecteur du Blog

Consignes

- Créer les tables users et posts avec les données de départ.
- Écrire lire_blog.py et afficher les titres avec leurs auteurs.
- Rechercher les identifiants 1 puis 999 avec fetchone().
- Rejouer la jointure avec psycopg2 sur PostgreSQL.
- Fermer les ressources et conserver les scripts Python.

Résultat attendu

- Trois articles s’affichent dans l’ordre des identifiants.
- Amina a deux articles, Paul en a un.
- L’identifiant 999 affiche un message adapté.
- Les deux moteurs produisent des lignes équivalentes.
- Le script se termine sans connexion laissée ouverte.

Erreurs fréquentes à repérer

- no such table : vérifier le chemin de blog.db et le schéma importé.
- Connection refused : vérifier le serveur PostgreSQL, l’hôte et le port.
- Authentification refusée : vérifier le compte, le mot de passe et ses droits.
- row vaut None : traiter le cas introuvable avant tout accès à row[0].
- Liste incomplète : vérifier si fetchone() a déjà consommé une ligne.

Méthode : lire le message, vérifier la connexion, puis isoler la requête.

Synthèse & ressources

À retenir

- Le pilote relie Python au moteur SQL.
- Une connexion ouvre une session avec la base.
- Le curseur exécute la requête et lit ses lignes.
- fetchone() renvoie une ligne ou None.
- fetchall() renvoie les lignes restantes.
- JOIN permet d’afficher les auteurs des articles.

Pour aller plus loin

Python : sqlite3
https://docs.python.org/3/library/sqlite3.html

Psycopg 2 : utilisation
https://www.psycopg.org/docs/usage.html

Psycopg 2 : connexion
https://www.psycopg.org/docs/module.html

Prochaine étape : insérer, modifier et supprimer des articles en sécurité.